REFLEX RUSH GAME DEVELOPMENT USING PYTHON

AIM

To design and develop an interactive reaction-time testing game using Python Tkinter that measures the user's reaction speed by recording the time taken to click a button when it turns green. The game displays reaction time, calculates the best and average scores, and provides restart functionality..

PROCEDURE

1. Start the program and import the required Python libraries: tkinter, random, and time.
2. Create the main application window using Tkinter and set its title as "REFLEX RUSH".
3. Design the graphical interface with a game title, instructions, round counter, action button, and score display.
4. Create a Start Game button to begin the reaction-time test.
5. Generate a random waiting time between 2 and 5 seconds using the random module.
6. Use Tkinter's after() method to display the green signal after the waiting period.
7. Record the signal time using time.perf_counter() when the button turns green.
8. Calculate the user's reaction time in milliseconds when the user clicks the green button.
9. Store the reaction times and repeat the test for five rounds.
10. Calculate and display the best and average reaction times using Python functions.
11. Provide a restart button to reset the game and allow the user to play again.
12. Execute the program and verify the random delay, reaction-time calculation, scoring, and restart functionality.

PROGRAM CODE

```
import tkinter as tk

import random

import time



class ReflexRush:

    def __init__(self, root):

        self.root = root

        self.root.title("REFLEX RUSH - Reaction Time Tester")

        self.root.geometry("520x440")

        self.root.resizable(False, False)

        self.root.configure(bg="#101827")


        self.waiting_job = None

        self.start_time = None

        self.round_number = 0

        self.reaction_times = []

        self.state = "ready"


        tk.Label(

            root,

            text="REFLEX RUSH",

            font=("Arial", 25, "bold"),

            bg="#101827",

            fg="#F8FAFC"

        ).pack(pady=(22, 3))


        tk.Label(

            root,
```

```
        text="How fast can you react?",
        font=("Arial", 12),
        bg="#101827",
        fg="#AAB8CC"
    ).pack(pady=(0, 16))

    self.round_label = tk.Label(
        root,
        text="ROUND 0 / 5",
        font=("Arial", 11, "bold"),
        bg="#101827",
        fg="#FBBF24"
    )
    self.round_label.pack(pady=4)

    self.signal_button = tk.Button(
        root,
        text="CLICK START GAME",
        font=("Arial", 16, "bold"),
        width=24,
        height=4,
        bg="#334155",
        fg="white",
        activebackground="#475569",
        activeforeground="white",
        relief="flat",
        command=self.button_clicked
    )
    self.signal_button.pack(pady=12)

    self.message_label = tk.Label(
```

```
        root,
        text="Press the button to begin.",
        font=("Arial", 11),
        bg="#101827",
        fg="#D8E1EE"
    )
    self.message_label.pack(pady=4)

    self.result_label = tk.Label(
        root,
        text="Best: -- ms     Average: -- ms",
        font=("Arial", 11, "bold"),
        bg="#101827",
        fg="#38BDF8"
    )
    self.result_label.pack(pady=7)

    self.restart_button = tk.Button(
        root,
        text="RESTART",
        font=("Arial", 10, "bold"),
        bg="#FBBF24",
        fg="#101827",
        relief="flat",
        padx=18,
        pady=7,
        command=self.restart_game
    )
    self.restart_button.pack(pady=8)

def button_clicked(self):
```

```
if self.state == "ready":
    self.begin_round()

elif self.state == "waiting":
    if self.waiting_job is not None:
        self.root.after_cancel(self.waiting_job)
        self.waiting_job = None

    self.state = "ready"
    self.signal_button.config(
        text="TOO EARLY!",
        bg="#EF4444",
        activebackground="#DC2626"
    )
    self.message_label.config(
        text="Wait for GREEN next time. Click to try again."
    )

elif self.state == "go":
    reaction_time = (
        time.perf_counter() - self.start_time
    ) * 1000

    self.reaction_times.append(reaction_time)
    self.round_number += 1
    self.state = "ready"

    self.round_label.config(
        text=f"ROUND {self.round_number} / 5"
    )
```

```
        self.signal_button.config(
            text=f"{reaction_time:.0f} ms",
            bg="#334155",
            activebackground="#475569"
        )

        self.update_results()

        if self.round_number >= 5:
            self.finish_game()
        else:
            self.message_label.config(
                text="Good! Click the button to start the next round."
            )
            self.signal_button.config(text="NEXT ROUND")

    def begin_round(self):
        self.state = "waiting"

        self.signal_button.config(
            text="WAIT FOR GREEN...",
            bg="#D97706",
            activebackground="#B45309"
        )

        self.message_label.config(
            text="Stay focused. Do not click yet!"
        )

        delay = random.randint(2000, 5000)
        self.waiting_job = self.root.after(
```

```
        delay, self.show_signal
    )

def show_signal(self):
    self.waiting_job = None

    if self.state != "waiting":
        return

    self.state = "go"
    self.start_time = time.perf_counter()

    self.signal_button.config(
        text="CLICK NOW!",
        bg="#16A34A",
        activebackground="#15803D"
    )

    self.message_label.config(
        text="GREEN! Click as quickly as possible!"
    )

def update_results(self):
    best_time = min(self.reaction_times)
    average_time = (
        sum(self.reaction_times) / len(self.reaction_times)
    )

    self.result_label.config(
        text=f"Best: {best_time:.0f} ms     "
            f"Average: {average_time:.0f} ms"
```

```
    )

def finish_game(self):
    average_time = (
        sum(self.reaction_times) / len(self.reaction_times)
    )
    best_time = min(self.reaction_times)

    self.state = "finished"

    self.signal_button.config(
        text="GAME COMPLETE",
        bg="#334155",
        activebackground="#475569",
        state="disabled"
    )

    self.message_label.config(
        text=f"Finished! Best: {best_time:.0f} ms | "
            f"Average: {average_time:.0f} ms"
    )

def restart_game(self):
    if self.waiting_job is not None:
        self.root.after_cancel(self.waiting_job)
        self.waiting_job = None

    self.start_time = None
    self.round_number = 0
    self.reaction_times = []
    self.state = "ready"
```

```
        self.round_label.config(text="ROUND 0 / 5")

        self.signal_button.config(
            text="CLICK START GAME",
            bg="#334155",
            activebackground="#475569",
            state="normal"
        )

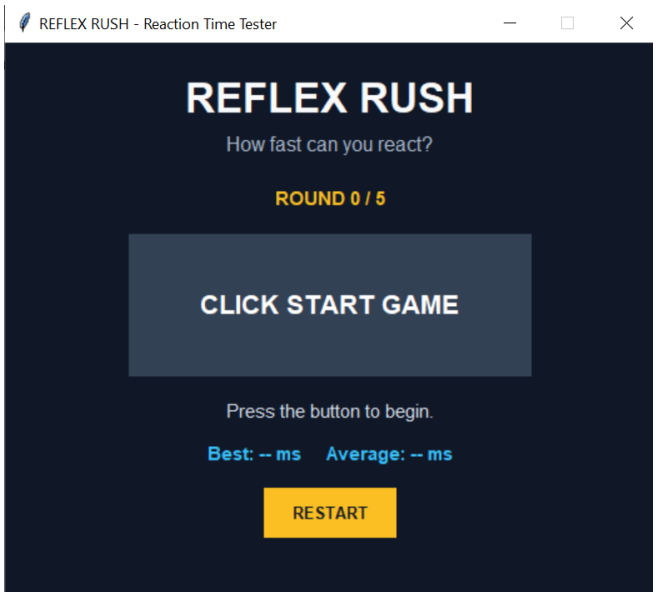
        self.message_label.config(
            text="Press the button to begin."
        )

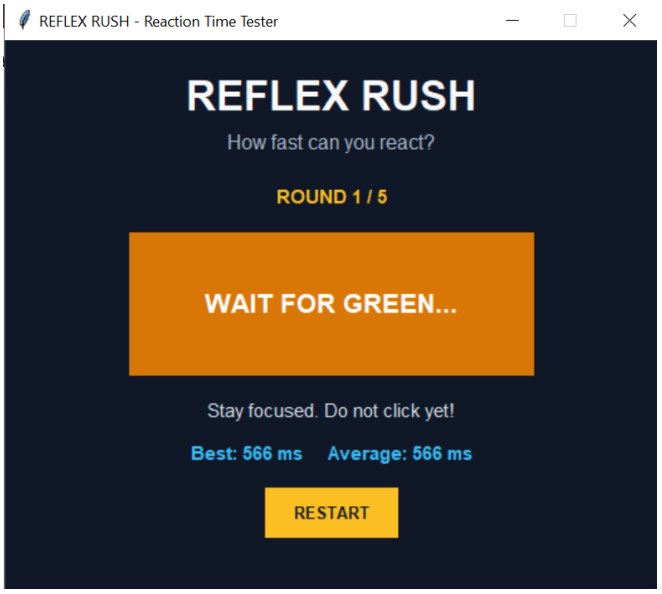
        self.result_label.config(
            text="Best: -- ms     Average: -- ms"
        )

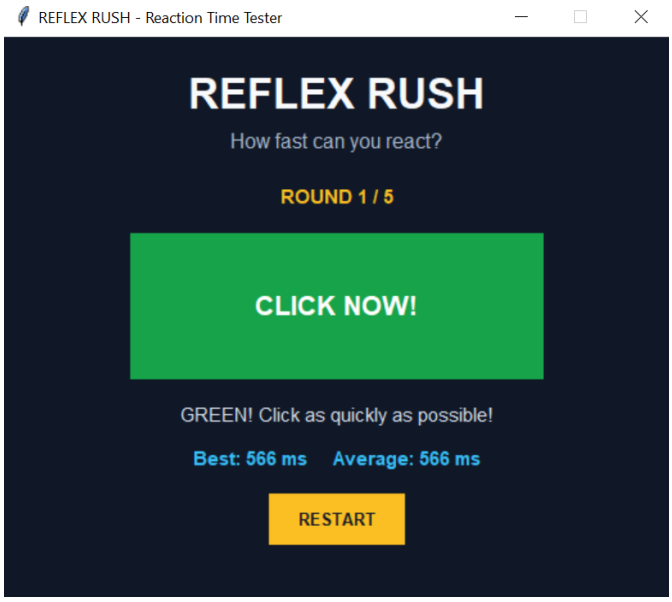


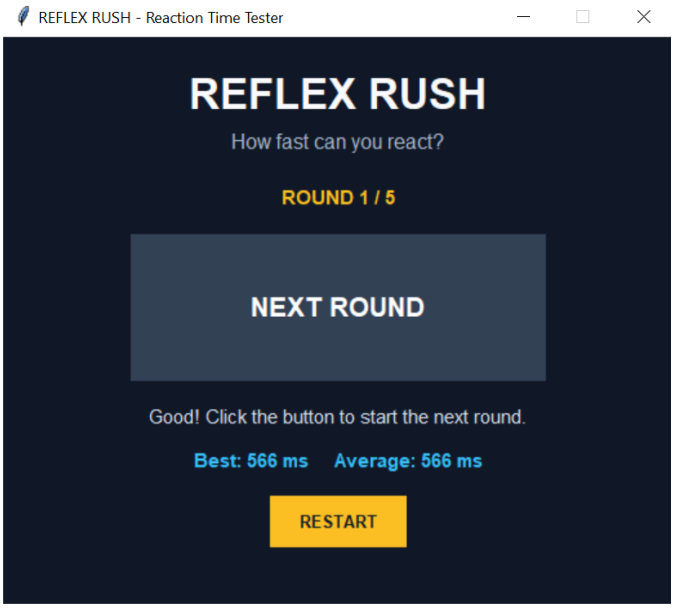
if __name__ == "__main__":
    window = tk.Tk()
    game = ReflexRush(window)
    window.mainloop()
```

output:









RESULT:

Thus, the Reflex Rush reaction-time testing game was successfully developed using Python Tkinter. The application measures the user's reaction time, displays the best and average reaction times, and provides restart functionality. The project demonstrates the use of Python libraries, functions, conditional statements, random number generation, and graphical user interface design.